

Advanced Amazon Lex Hands-on Task

Smart Customer Support Bot (Interview & Production Oriented)

Objective

Design and implement an advanced Amazon Lex V2 chatbot integrated with AWS Lambda, DynamoDB, and API Gateway. The bot should demonstrate production-grade conversational design, session management, persistence, and external API exposure.

Architecture Overview

Client (Web / Postman) → API Gateway → Lambda → Amazon Lex V2 → DynamoDB

AWS Services Used

- 1 Amazon Lex V2 – Conversational AI
- 2 AWS Lambda – Business logic and orchestration
- 3 Amazon DynamoDB – Persistent storage
- 4 Amazon API Gateway – External API exposure

Lex Bot – Intent Design

Intent 1: OpenSupportTicket

Slots: IssueType (Software/Hardware/Billing), IssueDescription, Urgency (Low/Medium/High). Use conditional slot elicitation, Lambda-based validation, and session attributes to track progress.

Intent 2: CheckTicketStatus

Slot: TicketId. Fetch ticket details dynamically from DynamoDB and respond with real-time status.

Intent 3: CancelTicket

Must interrupt any ongoing conversation. Lambda should override the active intent, clear slots, and update ticket status in DynamoDB.

Intent 4: Smart Fallback

Context-aware fallback that uses previous intent and last requested slot to ask clarifying questions and logs missed utterances.

Session & Context Management

Use Lex sessionAttributes and sessionState to store temporary context such as userId, currentFlow, lastQuestionAsked, and retryCount. Persist checkpoints in DynamoDB to resume conversations after session timeout.

DynamoDB Design

- 1 Table Name: SupportTickets
- 2 Partition Key: userId

- 3 Sort Key: ticketId
- 4 Attributes: issueType, description, urgency, status, createdAt, lastUpdated
- 5 Store conversation checkpoints for recovery

Lambda Responsibilities

- 1 Intent routing and dialog control
- 2 Slot validation and normalization
- 3 Read/write operations to DynamoDB
- 4 Intent switching and fallback handling
- 5 Business rules (e.g., high urgency escalation)

API Gateway Integration

Expose REST endpoints such as /chat/start, /chat/message, and /chat/status. Enable CORS and secure APIs using IAM or API Keys.

Edge Cases to Handle

- 1 User switches intent mid-conversation
- 2 Repeated invalid slot values
- 3 Session timeout and recovery
- 4 Duplicate ticket creation
- 5 Fallback escalation after multiple failures

How to Explain This in an Interview

This task demonstrates real-world Lex V2 usage with session recovery, intelligent fallback, Lambda-driven dialog control, and API-based integration suitable for enterprise applications.